

Rapport d'un projet personnel

TL – Traitement du Signal

Réalisé par :

Ahmed EL AYEB

Date : September 12, 2026

Contents

1	Partie 1: Quelques opérations de base sur les signaux	1
1.1	Signal numérique de synthèse	1
1.1.1	Génération du signal	1
1.1.2	Énergie et puissance	1
1.1.3	Quantification	2
1.2	Signal audio	3
1.2.1	Enregistrement	3
1.2.2	Restitution à différentes fréquences	3
1.2.3	Quantification du signal audio	4
1.2.4	Extraction et séparation de mots	4
2	Partie 2: Classification des signaux	4
2.1	Exemple de calcul théorique	4
2.2	Programmation	5
2.3	Application à la classification de quelques signaux simples	5
2.4	Classification de signaux de parole voisés ou non voisés	9
3	Partie 3: Aspects fréquentiels	11
3.1	Echantillonnage	11
3.2	La Transformée de Fourier discrète (TFD)	11
3.3	Changement de fréquence d'échantillonnage	13
3.4	Analyse spectrale	15
3.4.1	Analyse d'une tranche de signal par TFD	15
3.4.2	Effets de quelques fenêtres	18
4	Partie 4: Application : détection de pitch	22
4.1	Code python	22
4.1.0.1	Conclusion	24

1 Partie 1: Quelques opérations de base sur les signaux

1.1 Signal numérique de synthèse

1.1.1 Génération du signal

Un signal sinusoïdal de fréquence f_0 est généré par la fonction suivante:

$$x[n] = \sin\left(2\pi f_0 \frac{n}{f_e}\right)$$

où f_e est la fréquence d'échantillonnage et N le nombre d'échantillons.

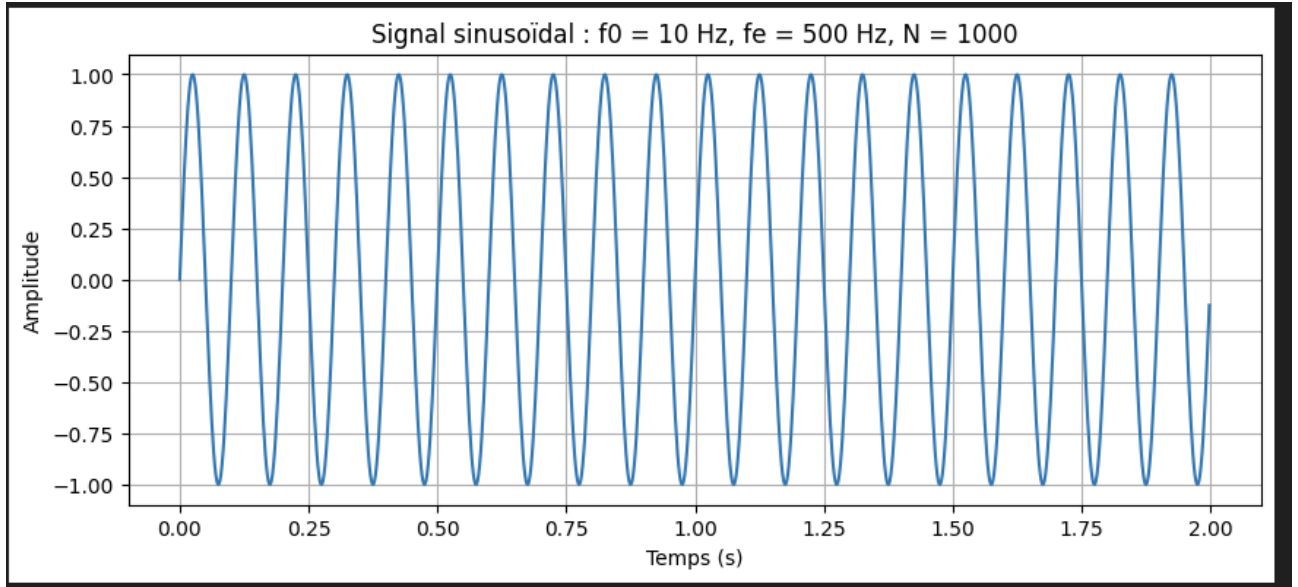


Figure 1: Signal sinusoïdal échantillonné

1.1.2 Énergie et puissance

L'énergie d'un signal discret $x[n]$:

$$E = \sum_{n=0}^{N-1} x[n]^2$$

Et la puissance moyenne :

$$P = \frac{1}{N} \sum_{n=0}^{N-1} x[n]^2$$

Considérons un signal sinusoïdal discret de la forme :

$$x[n] = A \cdot \sin\left(2\pi f_0 \frac{n}{f_e}\right)$$

La puissance moyenne théorique d'un signal périodique est calculée par la formule:

$$P = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^{N-1} x[n]^2$$

En utilisant l'identité trigonométrique :

$$\sin^2(\theta) = \frac{1 - \cos(2\theta)}{2}$$

on obtient :

$$x[n]^2 = A^2 \cdot \frac{1 - \cos\left(4\pi f_0 \frac{n}{f_e}\right)}{2}$$

Ainsi, la puissance devient :

$$P = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^{N-1} A^2 \cdot \frac{1 - \cos\left(4\pi f_0 \frac{n}{f_e}\right)}{2} = \lim_{N \rightarrow \infty} \frac{A^2}{2} - \frac{A^2}{2N} \sum_{n=0}^{N-1} \cos\left(4\pi f_0 \frac{n}{f_e}\right) = \frac{A^2}{2}$$

Donc dans notre cas la puissance moyenne théorique est égale à 0.5.

Pour la puissance moyenne calculée numériquement pour le signal échantillonné on a presque la même formule:

$$P = \frac{A^2}{2} - \frac{A^2}{2N} \sum_{n=0}^{N-1} \cos\left(4\pi f_0 \frac{n}{f_e}\right)$$

Disjonction de cas: - Si N est un multiple entier de la période alors:

$$\sum_{n=0}^{N-1} \cos(4\pi f_0 t) = 0 \Rightarrow P = \frac{A^2}{2}$$

- Sinon : la somme ne s'annule pas et on observe une légère déviation de la puissance par rapport à $\frac{A^2}{2}$. Cela est dû au fait que le signal est tronqué entre deux points non symétriques.

```
puissance moyenne du signal echantillonné : 0.5016748559921755
puissance moyenne du signal echantillonné : 0.5016748559921755
puissance moyenne du signal echantillonné : 0.5016748559921755
en variant N, c'est pas loin de la puissance moyenne theorique qui est évaluée à 0.5
```

Figure 2: Énergie et puissance du signal

1.1.3 Quantification

Quantification

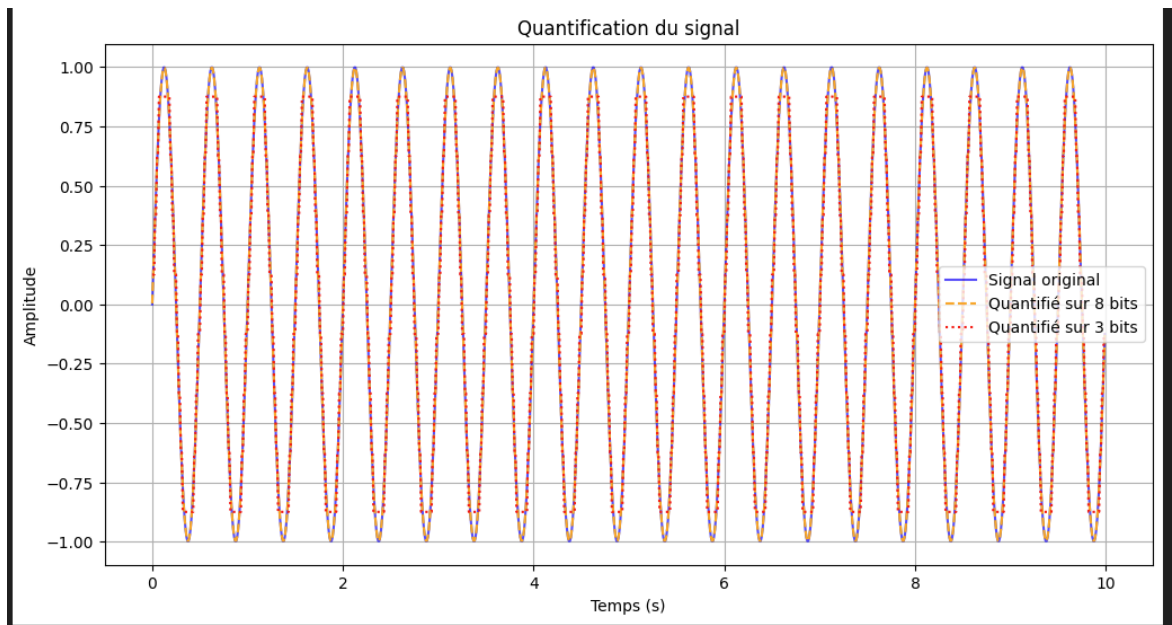


Figure 3: Quantification du signal à 3 et 8 bits

Pour la quantification à 3 bits, on retrouve bien 8 niveaux de quantification.

Le signal à 8 bits suit mieux la forme continue du signal d'origine mais on voit quand même quelques erreurs. À 3 bits, les marches sont plus visibles et le signal produit est significativement moins fidèle au signal d'origine.

SNR (Signal-to-Noise Ratio) :

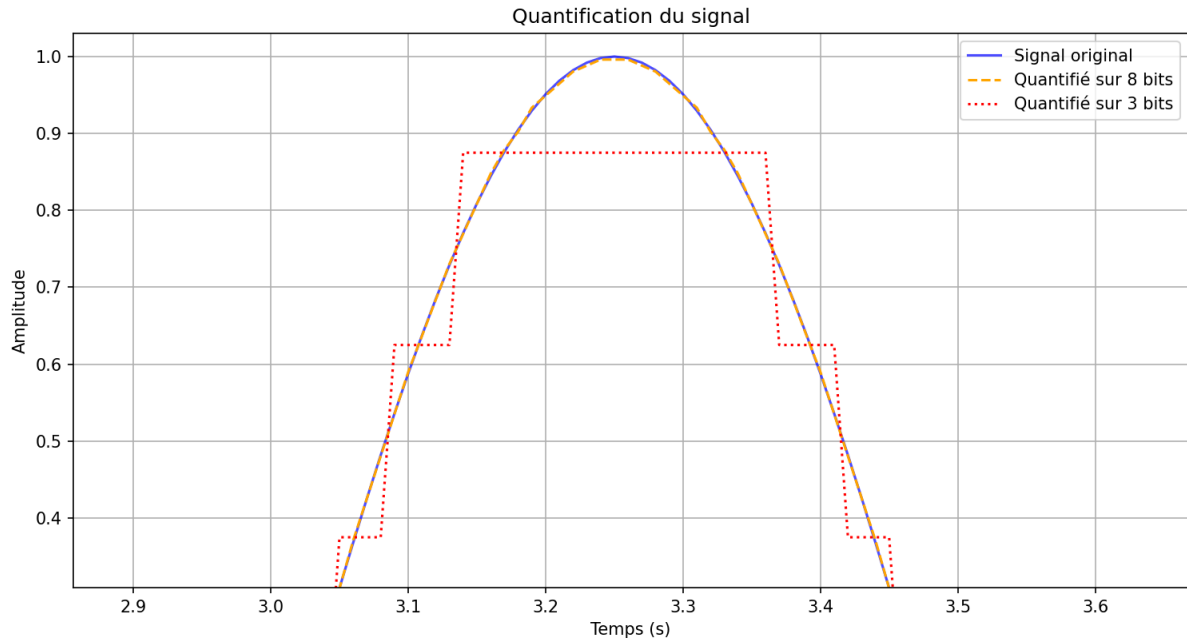


Figure 4: Zoom sur la quantification du signal à 3 et 8 bits

$$\text{SNR} = 10 \log_{10} \left(\frac{E_{\text{signal}}}{E_{\text{bruit}}} \right)$$

```
Énergie du bruit de quantification sur 8 bits: 0.005710
SNR 8 bits: 49,92 dB

Énergie du bruit de quantification sur 3 bits: 6.773188
SNR 3 bits: 19,82 dB
```

Figure 5: SNR pour chaque niveau de quantification

On remarque que l'énergie du bruit est plus élevée pour le signal quantifié à 3 bits. Le résultat est logique car on voit sur le graphe que ce signal est plus éloigné du signal d'origine comparé au signal quantifié à 8 bits. On a la même conclusion en raisonnant sur le SNR: $\text{SNR}_{q8} > \text{SNR}_{q3}$.

1.2 Signal audio

1.2.1 Enregistrement

Les mots « Chat » et « Chien » ont été enregistrés via Audacity CF figure 6).

1.2.2 Restitution à différentes fréquences

L'audio est lu à $f_e, 2f_e, 4f_e, \frac{f_e}{2}, \frac{f_e}{4}$.

Influence du changement de la fréquence :

- **Durée :** doubler la fréquence f_e divise la durée par deux et par conséquent accélère la voix , tandis que la diviser par deux double la durée ce qui ralentit la voix.
- **Hauteur :** multiplier la fréquence f_e rend la voix plus aiguë (fréquences doublées), tandis que la diminuer la rend plus grave (fréquences divisées).

Lorsque la fréquence de restitution est trop grande ou trop petite comparée à la fréquence d'échantillonnage, le son devient incompréhensible. Le son est trop rapide ou trop lent, mais aussi on ne reconnaît plus les mots. Cela s'explique notamment par le déplacement des formants, c'est-à-dire des pics de résonance caractéristiques des voyelles et consonnes, qui changent de position dans le spectre. Leur modification rend la parole méconnaissable, car ce sont eux qui permettent d'identifier les sons du langage humain.

1.2.3 Quantification du signal audio

- À 3 bits, le signal est très morcelé, ce qui altère fortement la qualité audio.
- À 8 bits, le son est encore reconnaissable, mais reste bizarre à écouter il y a trop de bruit mais moins que la quantification à 3bits.

1.2.4 Extraction et séparation de mots

Cf code python

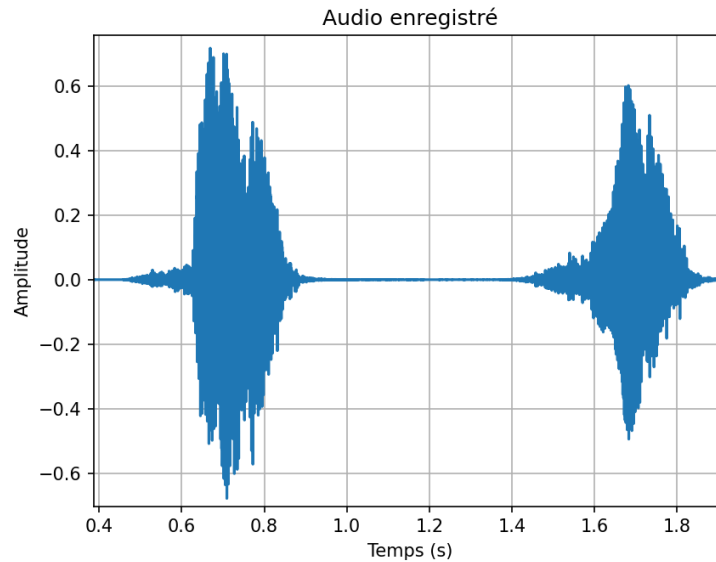
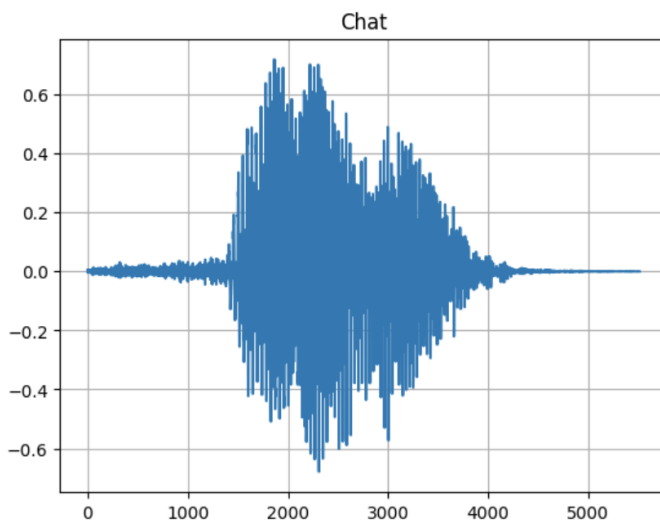
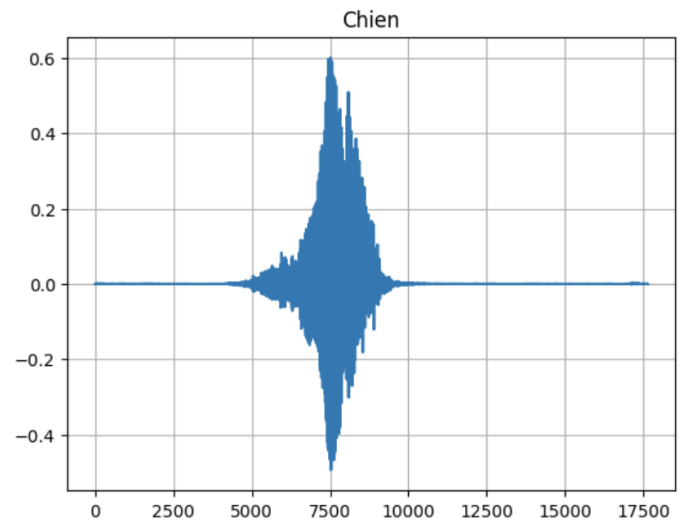


Figure 6: Signal audio enregistré



(a) Premier mot: "Chat"



(b) 2ème mot: "Chien"

Figure 7: Séparation des mots dans le signal audio

2 Partie 2: Classification des signaux

2.1 Exemple de calcul théorique

Soit un signal sinusoïdal discret défini par :

$$x[n] = A \sin(\omega n T_e) \quad \text{avec } \omega = 2\pi f \text{ pour alléger le calcul}$$

L'autocorrélation théorique du signal discret est définie par :

$$R_{xx}[k] = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^{N-1} x[n] \cdot x[n+k]$$

En remplaçant l'expression de $x[n]$:

$$\begin{aligned} R_{xx}[k] &= \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^{N-1} A \sin(\omega n T_e) \cdot A \sin(\omega(n+k)T_e) \\ &= A^2 \cdot \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^{N-1} \sin(\omega n T_e) \cdot \sin(\omega n T_e + \omega k T_e) \end{aligned}$$

En utilisant l'identité trigonométrique :

$$\sin(a) \sin(b) = \frac{1}{2} [\cos(a-b) - \cos(a+b)]$$

On a :

$$\begin{aligned} \sin(\omega n T_e) \cdot \sin(\omega n T_e + \omega k T_e) &= \frac{1}{2} [\cos(\omega k T_e) - \cos(2\omega n T_e + \omega k T_e)] \\ \Rightarrow R_{xx}[k] &= \frac{A^2}{2} \cos(\omega k T_e) \end{aligned}$$

car la somme de $\cos(2\omega n T_e + \cdot)$ sur une grande fenêtre N tend vers 0.

Conclusion : l'autocorrélation théorique du signal sinusoïdal discret est donnée par :

$$R_{xx}[k] = \frac{A^2}{2} \cos(\omega k T_e)$$

où :

- A est l'amplitude du signal,
- $\omega = 2\pi f$ est la pulsation du signal en rad/s,
- $T_e = \frac{1}{f_e}$ est la période d'échantillonnage,
- $k \in \mathbb{Z}$ est le décalage (lag) discret.

2.2 Programmation

Voici comment programmer cette fonction sous Python :

$$R_{xx}[k] = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \cdot x[n+k]$$

```
def autocorrelation_manual(x):
    N = len(x)
    l = np.arange(-N + 1, N)
    R = np.zeros(2*N - 1)

    for k in range(-N + 1, N):
        s = 0
        for n in range(N - abs(k)):
            s += x[n + k] * x[n] if k >= 0 else x[n] * x[n - k]
        R[k + N - 1] = s
    return l, R
```

Pour convertir les abscisses en secondes, on utilise la formule :

$$t = k.T_e = \frac{k}{f_e}$$

Et on compare avec l'autocorrélation calculée avec numpy et on dessine l'écart entre les résultats (manuelle et celle avec numpy) .

L'écart est de l'ordre de 10^{-12} entre les deux méthodes. Cela est grace à l'optimisation qui est derrière xcorr.

2.3 Application à la classification de quelques signaux simples

On génère les signaux demandés avec le code suivant :

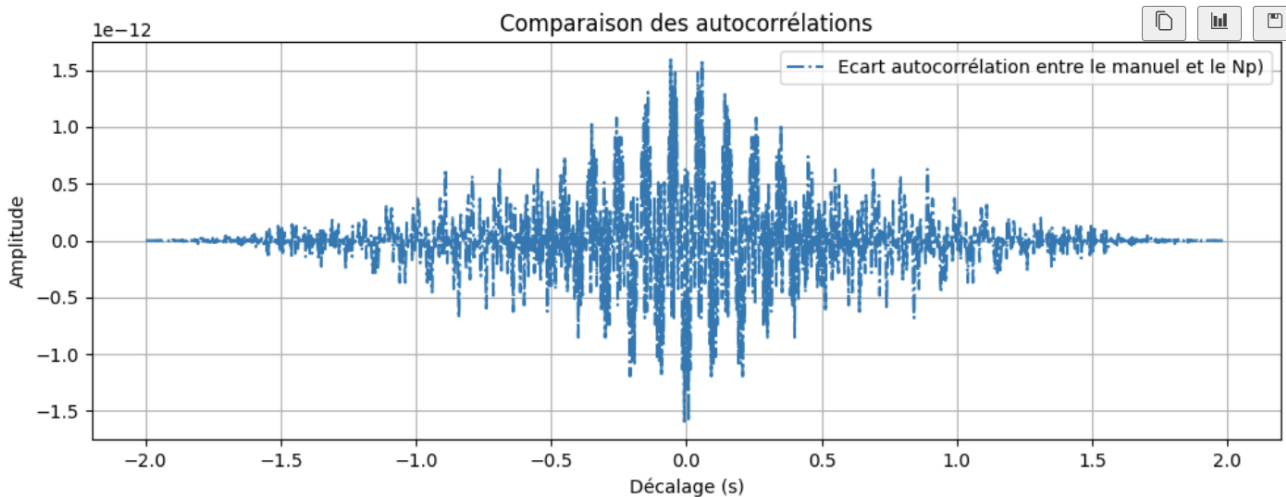


Figure 8: Ecart entre l'autocorrélation calculatoire et la fonction xcorr

```
fe = 11000
duration = 1#s
t = np.linspace(0, duration, int(fe * duration), endpoint=False)

f = 200
sinus = np.sin(2 * np.pi * f * t)

# 2. Signal triangulaire centré en 0 (série de Fourier avec 10 termes)
tri = np.zeros_like(t)
for k in range(1, 11):
    n = 2 * k - 1 # uniquement les harmoniques impaires
    tri += ((-1)**((k+1)) / n**2) * np.sin(2 * np.pi * n * f * t)
tri *= (8 / (np.pi**2)) # normalisation série de Fourier

# 3. Bruit blanc gaussien
taille = fe # 1 seconde
bruit = np.random.randn(taille)
bruit /= np.max(np.abs(bruit)) # normalisation

write("monbruit.wav", fe, bruit.astype(np.float32))
```

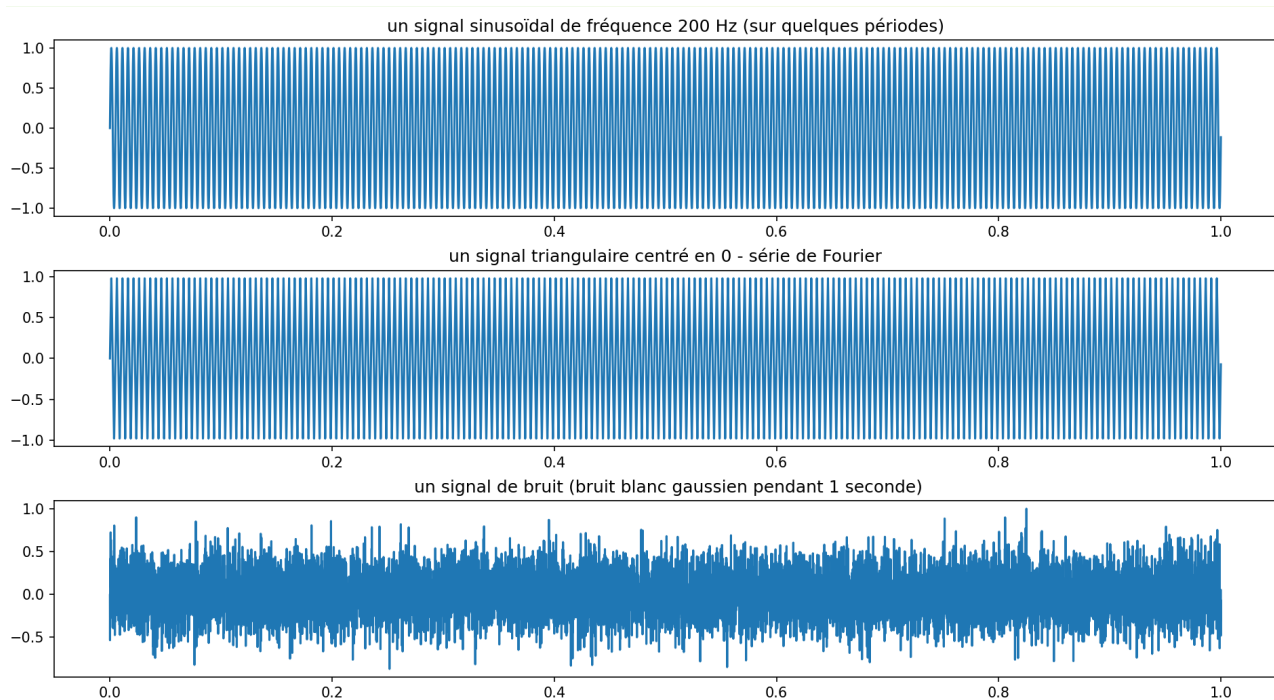


Figure 9: Signaux générés

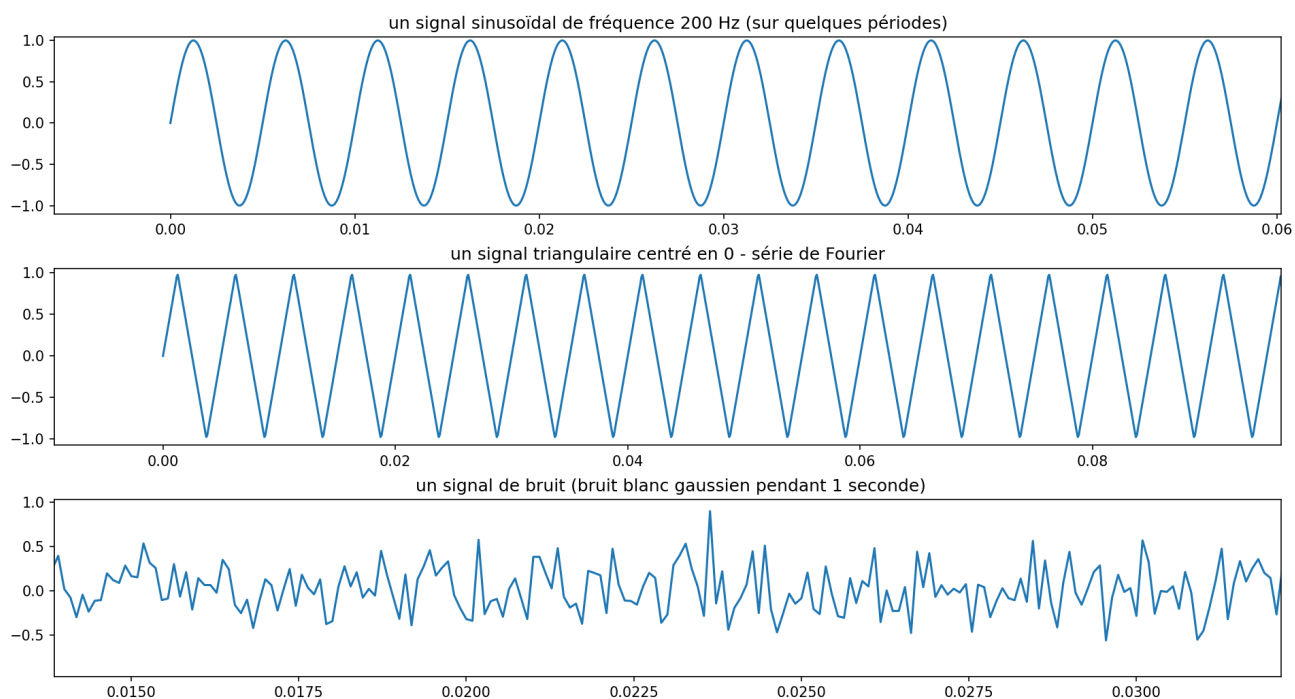


Figure 10: Signaux générés (zoom sur quelques périodes)

On a enregistré également sous Audacity la syllabe « aa » sur une durée d'environ 1 seconde et On calcule l'autocorrélation de plusieurs tranches du « aa » durant quelques dizaines de millisecondes l'autocorrélation pour chaque signal (30 ms). On trace également l'autocorrélation du signal enregistré sur deux tranches de 80ms à des moments différents (500ms et 800ms).

Définition d'un signal stationnaire: Un signal est dit **stationnaire** si ses propriétés statistiques, telles que la moyenne, la variance et l'autocorrélation, ne varient pas dans le temps.

L'analyse des autocorrélations montre que :

- Le signal vocal « aa » est **non stationnaire** car ses caractéristiques changent entre différentes tranches temporelles (voix humaine évolue dans le temps).

- Le signal de bruit blanc gaussien est **stationnaire**, ses propriétés statistiques étant constantes dans le temps (Pic au centre, décroissance rapide).
- Les signaux sinusoïdal et triangulaire, synthétisés sur quelques périodes, sont également **stationnaires** car ils sont périodiques et leurs statistiques ne varient pas dans le temps.

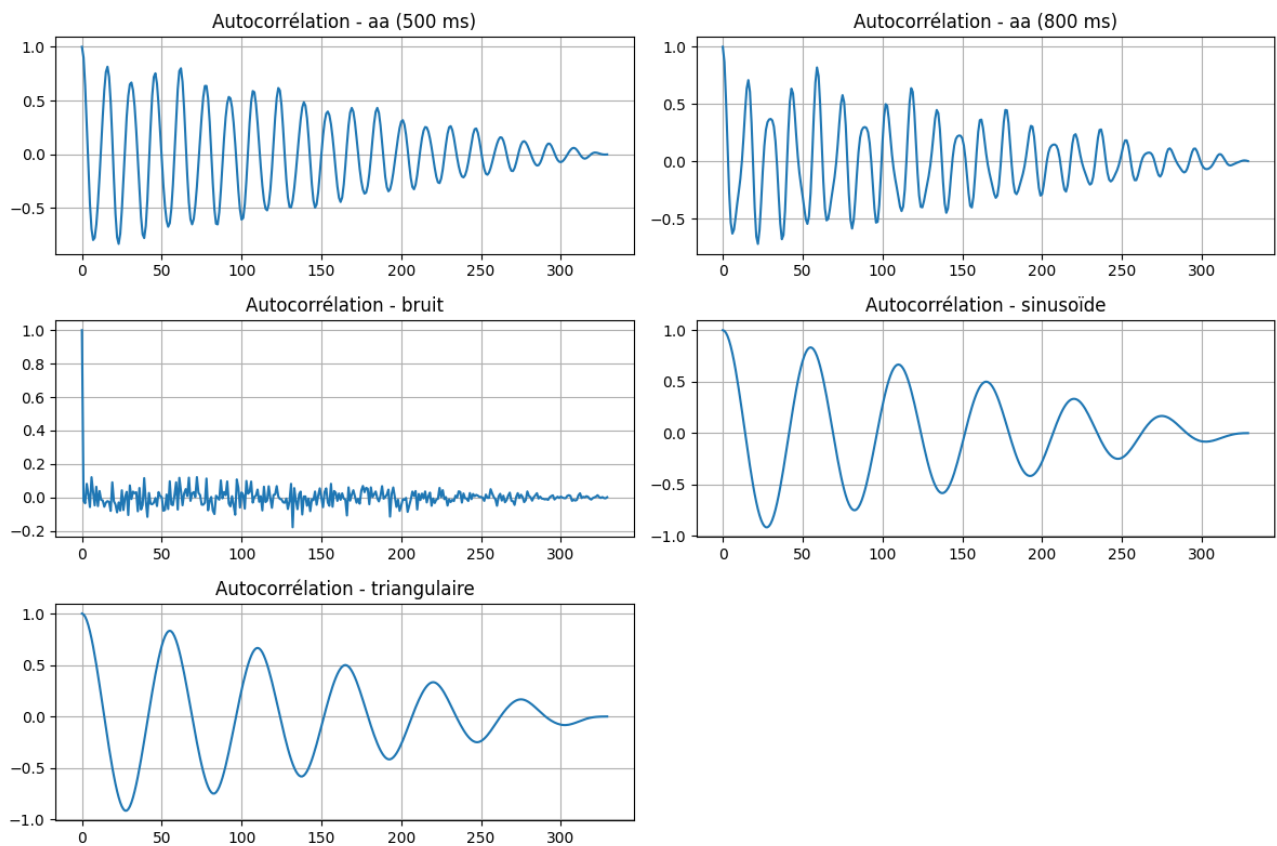


Figure 11: autocorrélation des signaux sur 30ms

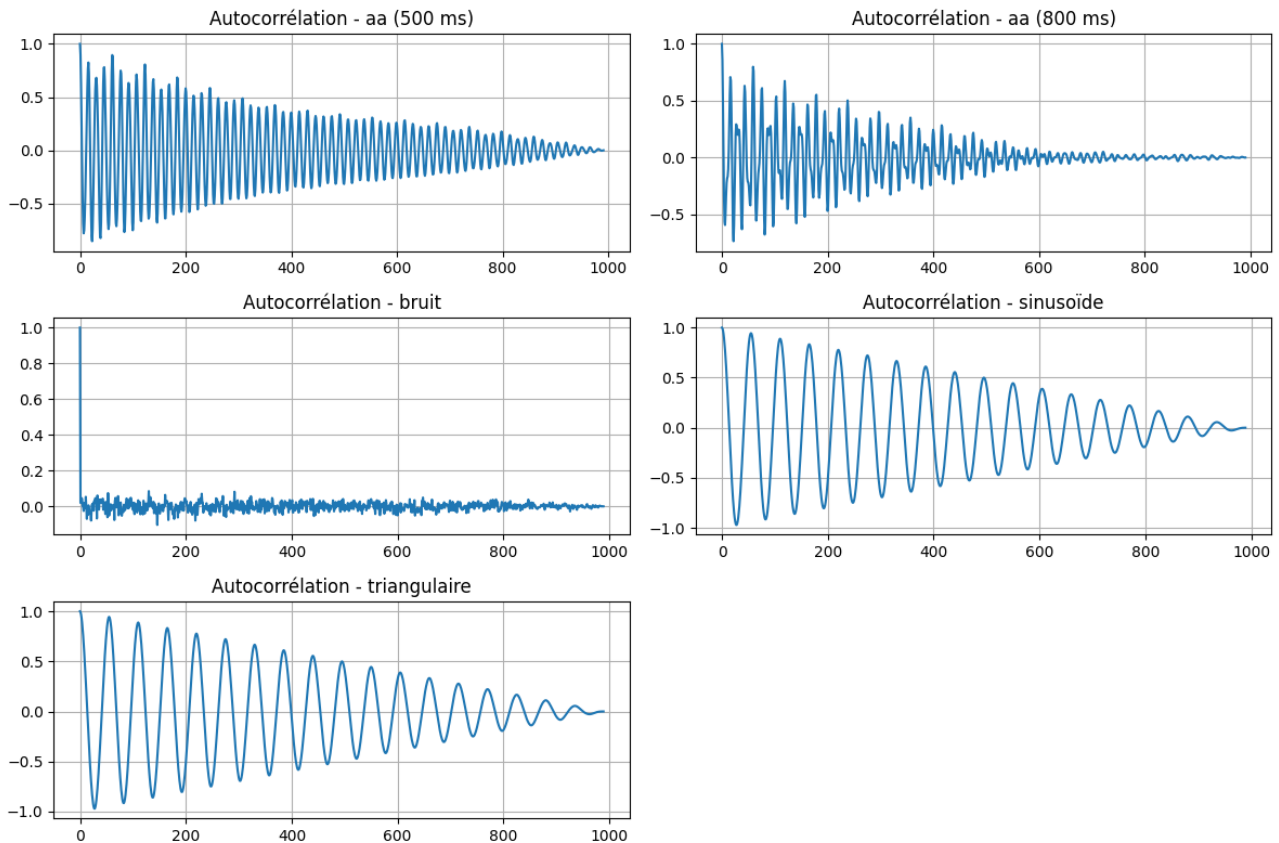


Figure 12: autocorrélation des signaux sur 90ms

2.4 Classification de signaux de parole voisés ou non voisés

Le signal est découpé en tranches de 30 ms pour analyser leur autocorrélation. Les résultats montrent clairement deux types de comportements :

- **Parties non voisées** (« chhhh ») : l'autocorrélation est bruitée, sans périodicité marquée, caractéristique des sons fricatifs.
- **Parties voisées** (« aaaa ») : l'autocorrélation présente des pics réguliers, révélant une structure périodique due à la vibration des cordes vocales.

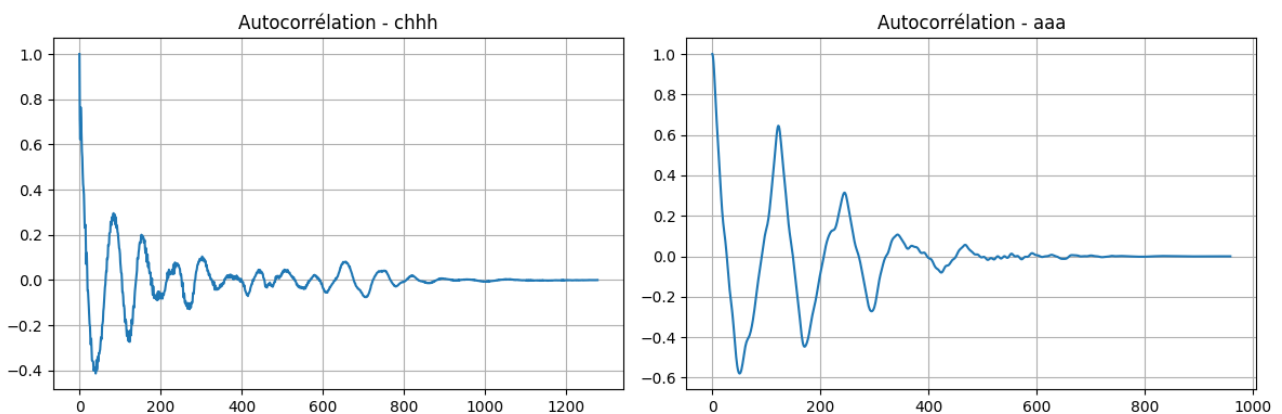


Figure 13: Autocorrélation de différentes tranches du mot « chat » : (gauche) non voisée, (droite) voisée.

L'autocorrélation permet d'estimer la fréquence fondamentale f_0 d'un signal voisé. On s'attend à ce qu'un signal périodique présente un maximum secondaire au retard correspondant à sa période fondamentale $T_0 = \frac{1}{f_0}$.

Pour identifier les pics secondaires dans l'autocorrélation, nous utilisons la fonction `find_peaks` de `scipy.signal`. Afin d'éviter de détecter des pics non significatifs dus au bruit ou à des résonances (formants), le paramètre **height** qui correspond à la hauteur minimale est fixé à 0,3 après normalisation pour ignorer les pics trop faibles dus au bruit.

Pour la tranche correspondant à la voyelle « *a* » dans l'enregistrement (partie voisée), nous obtenons après traitement:

- Indices du premier pic détecté : 122
- On en déduit une fréquence fondamentale :

$$f_0 = \frac{f_e}{\text{lag}} = \frac{16\,000}{122} \approx 131,3 \text{ Hz}$$

Ce résultat est conforme aux plages typiques pour une voix masculine (50–150 Hz).

3 Partie 3: Aspects fréquentiels

3.1 Echantillonnage

Le théorème de Shannon stipule qu'un signal bande limitée, dont la fréquence maximale est f_0 , peut être parfaitement reconstruit à partir de ses échantillons si la fréquence d'échantillonnage vérifie (au moins) :

$$f_e > 2f_0$$

La fréquence $f_e = 2f_0$ est appelée **fréquence de Nyquist** et constitue la limite minimale pour éviter le phénomène de repliement spectral (*aliasing*).

On échantillonne donc le signal selon trois cas distincts :

- **Cas 1** : $f_e = 2f_0$ (fréquence de Nyquist),
- **Cas 2** : $f_e = 5f_0$ (fréquence supérieure),
- **Cas 3** : $f_e = 0,8f_0$ (fréquence inférieure).

On voit que pour les deux premiers cas, le signal échantillonné est fidèle à la forme du signal d'origine. En revanche, pour le troisième cas, l'échantillonnage à une fréquence inférieure à la fréquence de Nyquist entraîne un repliement spectral, rendant le signal méconnaissable.

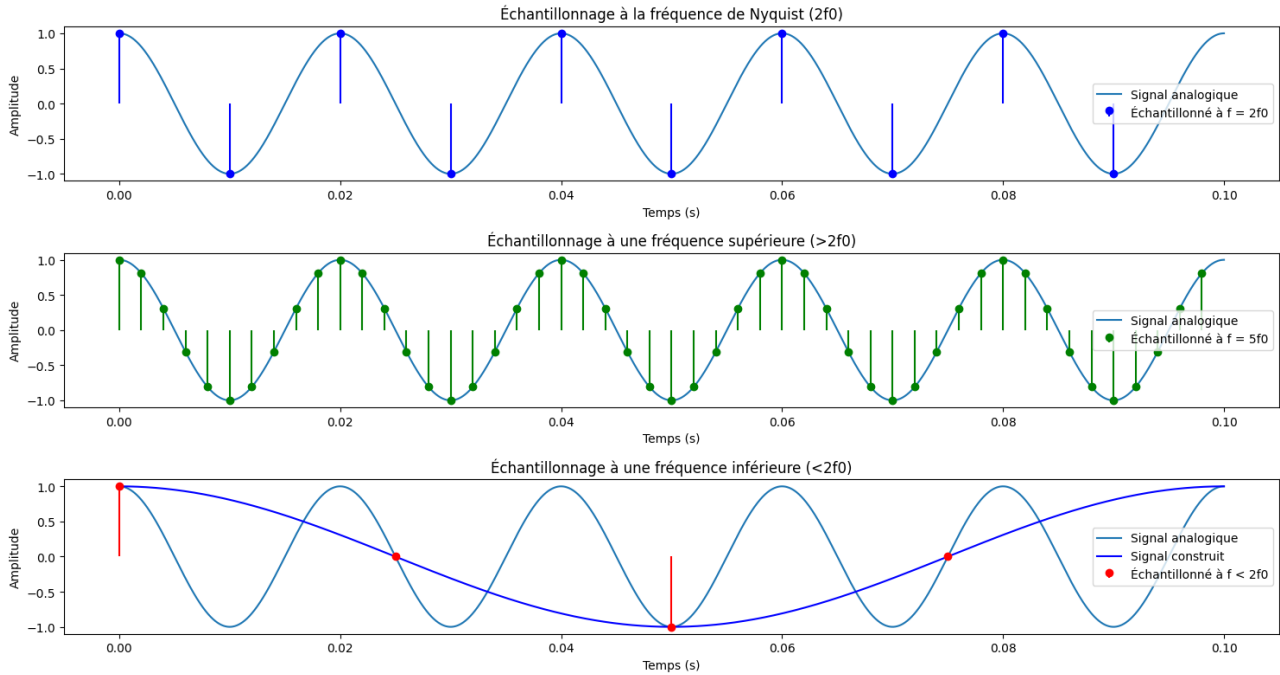


Figure 14: Signaux échantillonnés

3.2 La Transformée de Fourier discrète (TFD)

Nous avons implémenté une fonction en `Python` permettant de calculer et tracer la densité spectrale d'énergie d'un signal extrait d'un fichier `.wav`. La fonction lit le fichier audio, extrait une tranche du signal à partir d'un indice donné, applique une fenêtre de Hanning pour atténuer les effets de bord, puis calcule la transformée de Fourier rapide (FFT) sur 2^n points. La densité spectrale est obtenue en prenant le module au carré des coefficients de la FFT (représentant l'énergie en fréquence), normalisée et convertie en décibels (dB) selon la formule :

$$P_{dB}(f) = 10 \log_{10} \left(\frac{|FFT(f)|^2}{\max(|FFT(f)|^2)} + \varepsilon \right),$$

avec ε un terme petit (10^{-12}) pour éviter les problèmes numériques liés au logarithme de zéro.

La courbe obtenue est symétrique par rapport à l'axe des fréquences, car la FFT calcule les coefficients complexes pour les fréquences positives et négatives. On ne garde que la moitié positive du spectre.

Le choix de 2^n (taille de la FFT) influence la **résolution fréquentielle**. Plus n est grand, plus les fréquences sont discrétisées finement, ce qui permet de mieux isoler les pics correspondant aux composantes harmoniques du signal.

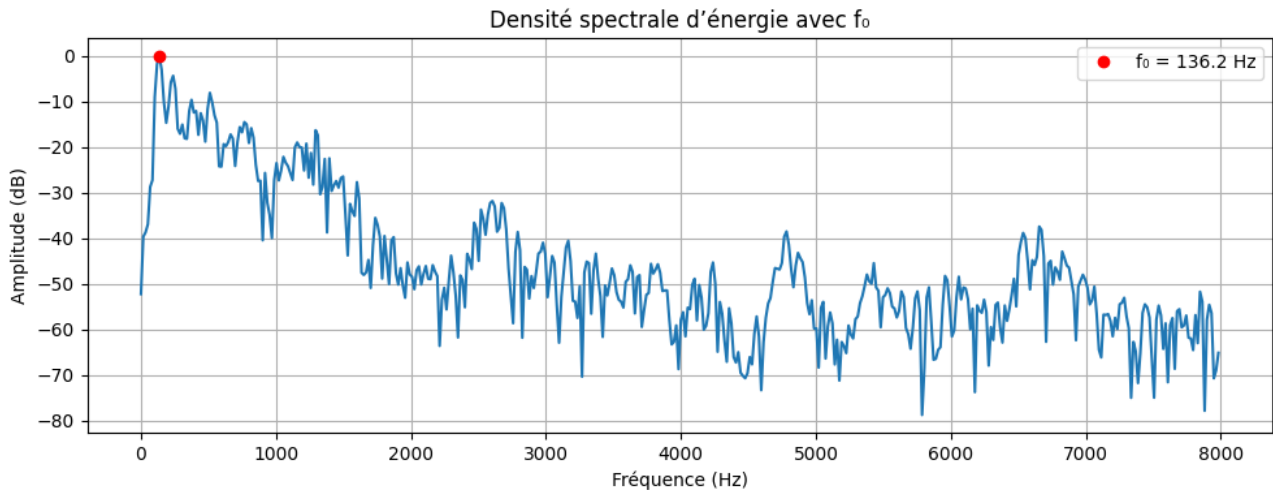


Figure 15: FFT

Sur le spectre obtenu, on observe une structure en pics caractéristiques d'un signal périodique voisé (ici le phonème *aaa*). La fréquence fondamentale correspond au premier pic significatif au-dessus du bruit. À partir du spectre, nous détectons une fréquence fondamentale de **136,2 Hz**.

Ce résultat est très proche de celui obtenu par l'autocorrélation dans le domaine temporel, où nous avons trouvé une fréquence de **133,3 Hz**. Cette légère différence peut s'expliquer par :

- la résolution fréquentielle limitée de la FFT (dépendante de la taille de la tranche),
- les effets de la fenêtre de Hanning qui modifie légèrement le contenu fréquentiel,
- et les arrondis ou approximations dans la localisation du pic.

Malgré cela, les deux méthodes convergent vers une estimation très cohérente de la fréquence fondamentale, typique d'une voix d'homme (comprise entre 50 et 150 Hz).

Le *zero-padding* consiste à compléter un signal par des zéros à la fin. Cette opération n'ajoute aucune nouvelle information, mais elle permet d'augmenter artificiellement la longueur du signal, ce qui améliore la **résolution fréquentielle** du spectre après transformation de Fourier.

Nous avons implémenté une fonction `zero_padding` en `Python`, prenant en entrée le signal initial et le nombre de zéros à ajouter, et renvoyant le signal étendu.

```
def zero_padding(signal, nb_zeros):
    return np.hstack((signal, np.zeros(nb_zeros)))
```

Nous avons ensuite comparé les densités spectrales d'énergie (obtenues par FFT) avant et après *zero-padding*. Le résultat montre que :

- les **positions des pics** dans le spectre restent inchangées (le contenu fréquentiel est le même),
- mais les courbes sont **plus lisses** comme on peut le voir dans la figure ?? et les pics **mieux localisés** après padding, ce qui facilite l'estimation des fréquences fondamentales,
- le *zero-padding* agit donc comme une **interpolation en fréquence**.

Cela est particulièrement utile pour distinguer des pics proches ou évaluer plus précisément une fréquence fondamentale.

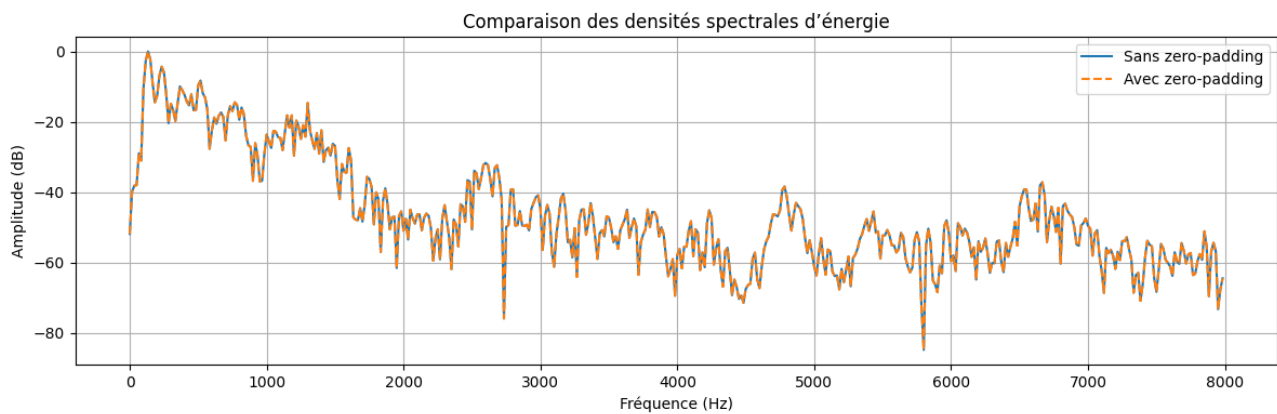


Figure 16: FFT avec et sans zero-padding

On peut faire directement du zero-padding dans la fonction `np.fft.fft(x, n)`.

- Si n est supérieur à la longueur de x , la fonction ajoute des zéros à la fin de x jusqu'à atteindre la longueur n .
- Si n est inférieur ou égal à la longueur de x , la fonction tronque x à la longueur n .

3.3 Changement de fréquence d'échantillonnage

Définitions:

- **la variation de la cadence (changer la vitesse de lecture)** consiste à lire un signal audio plus vite ou plus lentement, sans modifier son contenu en échantillons. Cela revient à modifier la *fréquence d'échantillonnage effective* sans changer les données elles-mêmes. Par exemple :
 - Lire un signal échantillonné à 44,100 Hz comme s'il était à 22,050 Hz revient à le lire deux fois plus lentement (et donc plus grave).
 - Inversement, le lire à 88,200 Hz le rend deux fois plus rapide (et plus aigu).
- **Réduction de cadence (ralentissement)** : on insère des échantillons ou on répète les échantillons existants. Cela allonge la durée du signal et rend la voix plus grave.
- **Augmentation de cadence (accélération)** : on sous-échantillonne le signal, par exemple en prenant un échantillon sur deux, ce qui réduit la durée et rend la voix plus aiguë.

Dans les deux cas, si l'on garde la même fréquence d'échantillonnage f_e , alors la durée du signal est directement modifiée.

Le facteur n de changement de cadence détermine le degré de ralentissement ou d'accélération :

- Si $n > 1$, on applique un ralentissement. Le signal devient plus lent, plus long et plus grave.
- Si $n < 1$ (ou bien $n = 1/k$ avec $k > 1$), on accélère le signal. Il devient plus court et plus aigu.

Par exemple :

- Pour $n = 2$, chaque échantillon est répété deux fois $\Rightarrow$ la durée est doublée.
- Pour $n = 1/3$ ($k = 3$), on garde un échantillon sur trois $\Rightarrow$ la durée est divisée par 3.

On utilise le code suivant :

```
def reduce_cadence(filename, tranche_size, start_index, factor, out_filename):
    # on acc l re le signal d'un facteur

    fs, signal = wavfile.read(filename)
    # On extrait la tranche demand e
    tranche = signal[start_index:start_index + tranche_size]
    # On d cime : on prend un chantillon sur 'factor'
```

```

reduced = tranche[::factor]
# On crit le resultat
wavfile.write(out_filename, fs, reduced.astype(np.int16)) # On garde la m me
fr quence d' chantillonnage

return reduced, fs

def increase_cadence(filename, tranche_size, start_index, factor, out_filename):
    # on va ralentir le signal d'un facteur 'factor' :
    # On r p te chaque chantillon 'factor' fois (np.repeat).
    # On conserve fs identique (donc la dur e est multipli e par factor).

    fs, signal = wavfile.read(filename)
    # On extrait la tranche demand e
    tranche = signal[start_index:start_index + tranche_size]
    # On r p te chaque chantillon factor fois
    upsampled = np.repeat(tranche, factor)
    # On crit le resultat
    wavfile.write(out_filename, fs, upsampled.astype(np.int16)) # On garde la m me
    fr quence d' chantillonnage

    return upsampled, fs

def generate_cadence_variations(filename, tranche_size, start_index):
    """
    G n re plusieurs fichiers .wav acc l r s (x2, x3, x4) et ralentis ( 2 , 3 ,
    4 )
    dans le dossier 'outputs_cadence/'. Pour chaque facteur f :
    - acc l r : fichier "faster_x{f}.wav"
    - ralenti : fichier "slower_x{f}.wav"
    """
    acceleration_factors = [2, 3, 4, 5, 6] # 2x, 3x, 4x plus rapides
    slowing_factors = [2, 3, 4, 5, 6] # 2 , 3 , 4 plus lents

    for f in acceleration_factors:
        out_file = f"outputs_cadence/faster_x{f}.wav"
        reduce_cadence(filename, tranche_size, start_index, f, out_file)
        print(f"[ ] Fichier g n r : {out_file} (acc l r x{f})")

    for f in slowing_factors:
        out_file = f"outputs_cadence/slown_x{f}.wav"
        increase_cadence(filename, tranche_size, start_index, f, out_file)
        print(f"[ ] Fichier g n r : {out_file} (ralenti {f})")

generate_cadence_variations(filename= "audio-sig.wav", tranche_size= 25000,
start_index = 500)

```

Les variations extrêmes de vitesse de lecture (facteur n très grand ou très petit) ont un effet significatif sur la perception du signal vocal. Lorsque le signal est fortement **accéléré** ($n \gg 1$), la durée devient très courte, les mots sont comprimés et la hauteur tonale augmente au point que la voix semble artificielle et difficilement reconnaissable. À l'inverse, un **ralentissement prononcé** ($n \ll 1$) allonge excessivement la durée du signal, donnant une impression de lenteur extrême ; les syllabes s'étalent et deviennent floues. Ainsi, au-delà de certains seuils critiques (typiquement $n \geq 3$ ou $n \leq 0.33$), la compréhension du message vocal devient très altérée. Cette dégradation est liée au déplacement ou à la déformation des formants, qui sont des indices essentiels à la reconnaissance des voyelles et de l'intelligibilité de la parole.

L'analyse fréquentielle à travers la transformée de Fourier des différentes versions du signal met en évidence ces altérations (voir figure 17) : **Signal original** : on observe un spectre de référence contenant des pics bien définis correspondant aux formants vocaux. Ces pics traduisent la structure acoustique propre à la voix humaine. **Accélération (x2, x4)** : les composantes fréquentielles sont déplacées vers les hautes fréquences. Les pics de formants se décalent vers la droite dans le spectre, ce qui traduit une élévation de la hauteur (pitch) de la voix. **Ralentissement (x2, x4)** : le spectre est

au contraire comprimé vers les basses fréquences. Cela entraîne un déplacement des formants vers la gauche, associé à une voix plus grave. Ce phénomène s'explique par l'ajout d'échantillons nuls, ce qui étire artificiellement le signal dans le temps. **Évolution de l'amplitude** : en accélérant, la perte d'échantillons réduit l'énergie totale du signal, entraînant une diminution de l'amplitude spectrale. Lors du ralentissement, l'insertion de z

Contrairement aux méthodes naïves d'augmentation ou de réduction de cadence par décimation (suppression d'échantillons) ou zéro-padding (ajout de zéros), la fonction `resample` de SciPy utilise une interpolation efficace basée sur la transformée de Fourier pour reconstruire le signal. Cette approche conserve bien mieux les caractéristiques spectrales du signal, notamment les formants vocaux.

- **À l'écoute**, les signaux transformés avec `resample` sont plus fluides, plus naturels, et conservent mieux l'intelligibilité du contenu vocal, même pour des facteurs élevés.
- En comparaison, la première méthode introduit des distorsions, un effet robotique ou haché, et altère la hauteur et le timbre de manière plus brutale.

3.4 Analyse spectrale

3.4.1 Analyse d'une tranche de signal par TFD

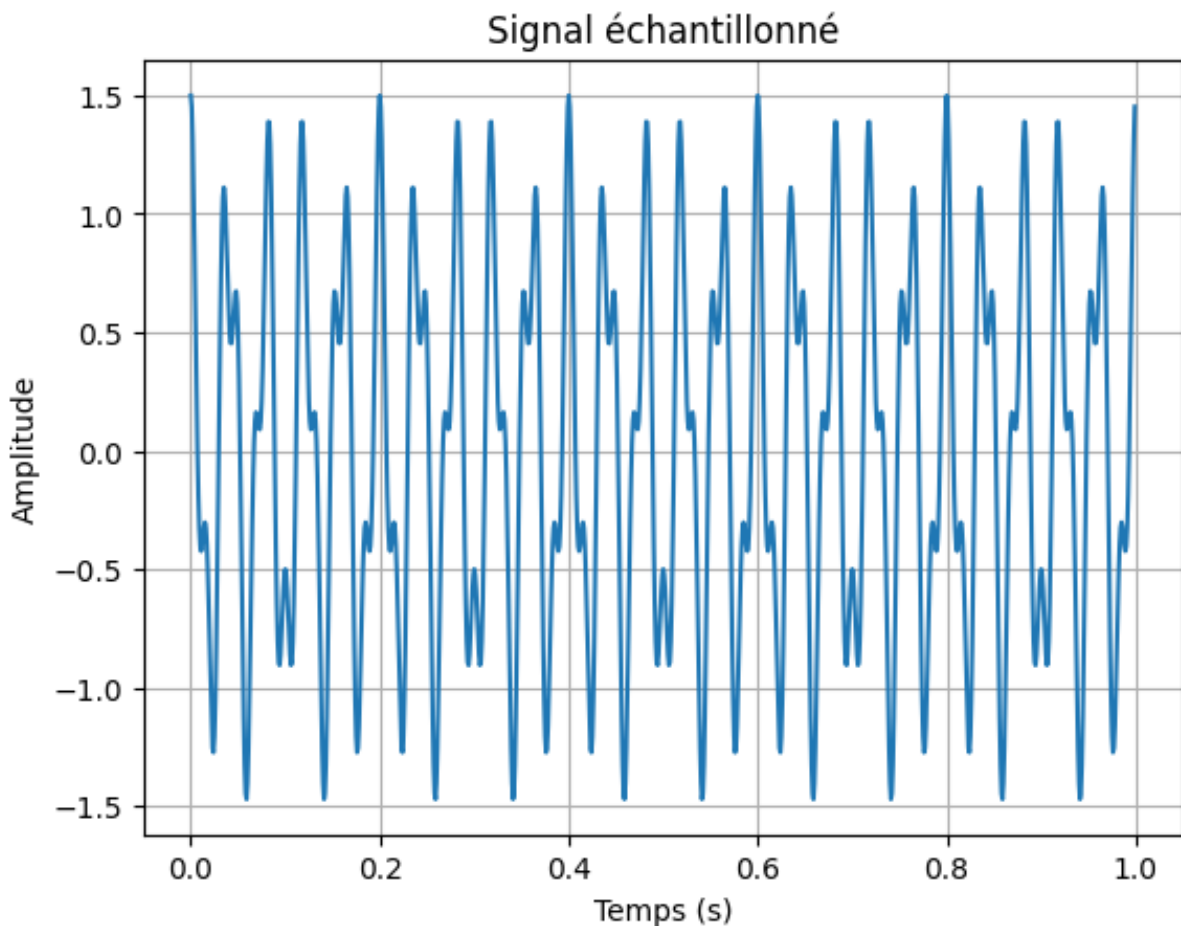


Figure 18: exemple d'un signal généré avec: $N = 1000$; $f_e = 1000$; $f_0 = 25$; $f_1 = 60$; $A_0 = 1.0$; $A_1 = 0.5$

La transformée de Fourier discrète d'une fenêtre de Hanning de longueur N présente un module en forme de triangle centré autour de la fréquence zéro. Par analyse expérimentale :

- Le lobe principal est bien approximé par une forme triangulaire.
- La largeur de ce lobe est d'environ $\frac{4}{N}$ en fréquence normalisée (de $-2/N$ à $+2/N$).
- Après normalisation, la surface sous ce lobe principal (approximation de l'intégrale) est proche de 1, ce qui montre que la majorité de l'énergie spectrale est concentrée dans cette bande.

Cela justifie l'utilisation fréquente de la fenêtre de Hanning dans les analyses spectrales pour limiter les effets de repliement spectral tout en gardant une bonne localisation fréquentielle.

Lorsque la longueur N de la fenêtre de Hanning augmente, on observe que le module de sa transformée de Fourier devient plus "fin". Cela s'explique par le compromis temps-fréquence inhérent à la transformée de Fourier : une plus grande durée d'observation (fenêtre plus longue) permet une meilleure résolution fréquentielle. Ainsi :

- Le lobe principal devient plus étroit, avec une largeur approximative de $\frac{4}{N}$.
- Le spectre conserve sa forme triangulaire mais est plus concentré autour de la fréquence zéro.
- L'amplitude du lobe principal augmente légèrement pour compenser la réduction de largeur (la surface reste ≈ 1).

Cela confirme que la fenêtre de Hanning agit comme un filtre passe-bas doux, et que plus elle est longue, plus elle isole finement une bande de fréquences autour de zéro.

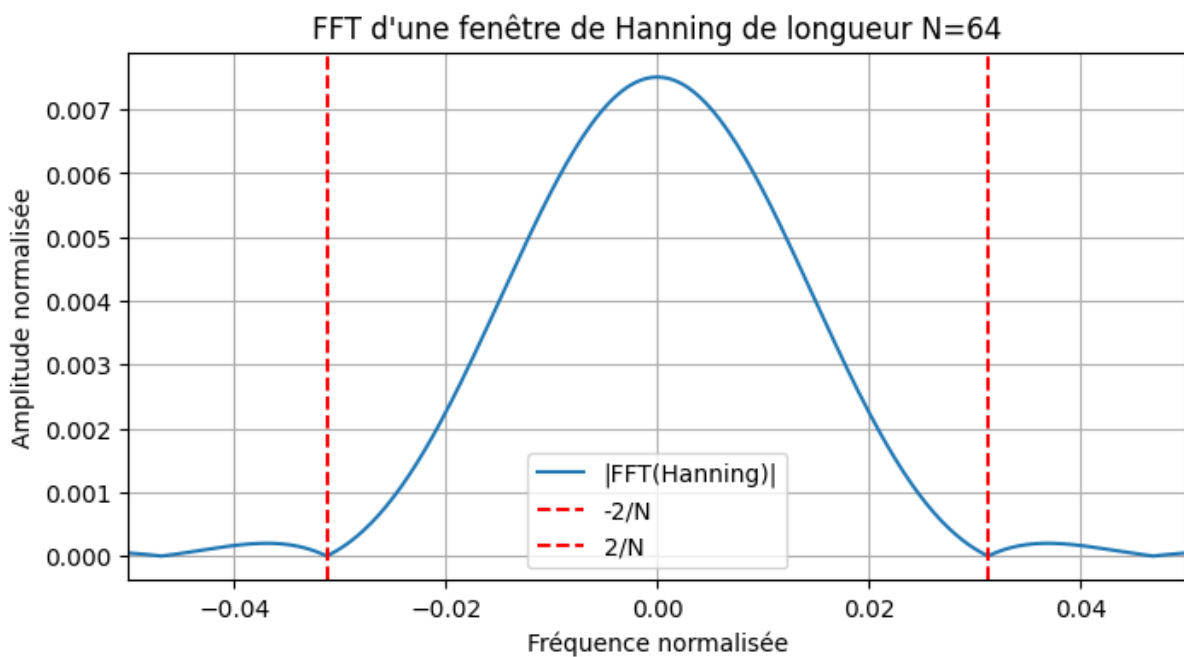


Figure 19: FFT fenêtre de Hanning $N = 64$

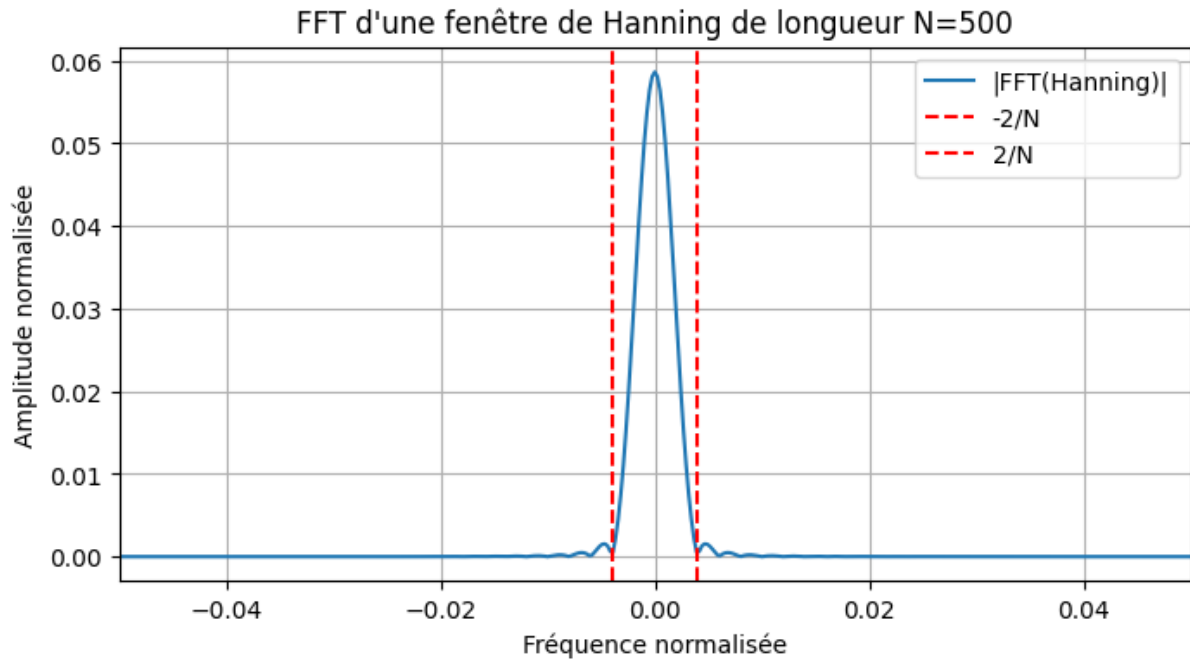


Figure 20: FFT fenêtre de Hanning $N = 1000$

On étudie le signal suivant :

$$x(t) = A \cos(2\pi f_0 t) + A \cos(2\pi f_1 t)$$

où $f_0 = 40$ kHz, $f_1 = 61$ kHz, $f_e = 512$ kHz et $A = 1$. Le signal est discrétisé, puis une fenêtre de $N = 256$ échantillons est sélectionnée pour effectuer la transformée de Fourier discrète (TFD), après application d'une fenêtre de Hanning.

Lorsqu'une sinusoïde est pondérée par une fenêtre de Hanning, sa TFD correspond à la convolution du spectre idéal (un pic unique) avec le spectre de la fenêtre (un triangle de surface unité et de largeur $\frac{4}{N}$).

Chaque composante sinusoïdale génère ainsi un lobe triangulaire :

- centré respectivement sur f_0 et f_1 ;
- de largeur en fréquence réelle donnée par :

$$\Delta f = \frac{4f_e}{N} = \frac{4 \cdot 512000}{256} = 8000 \text{ Hz}$$

Le spectre obtenu est donc constitué de la superposition de deux lobes triangulaires :

- chacun d'une largeur de 8 kHz ;
- avec une surface unitaire (due à la normalisation de la fenêtre).

Le résultat spectral met en évidence deux pics élargis autour de 40 kHz et 61 kHz, permettant d'identifier clairement les deux fréquences dominantes du signal, bien que la fenêtre limite la résolution.

L'utilisation de la fenêtre de Hanning permet d'atténuer efficacement les fuites spectrales, ce qui améliore la lisibilité des pics par rapport à une TFD sans pondération.

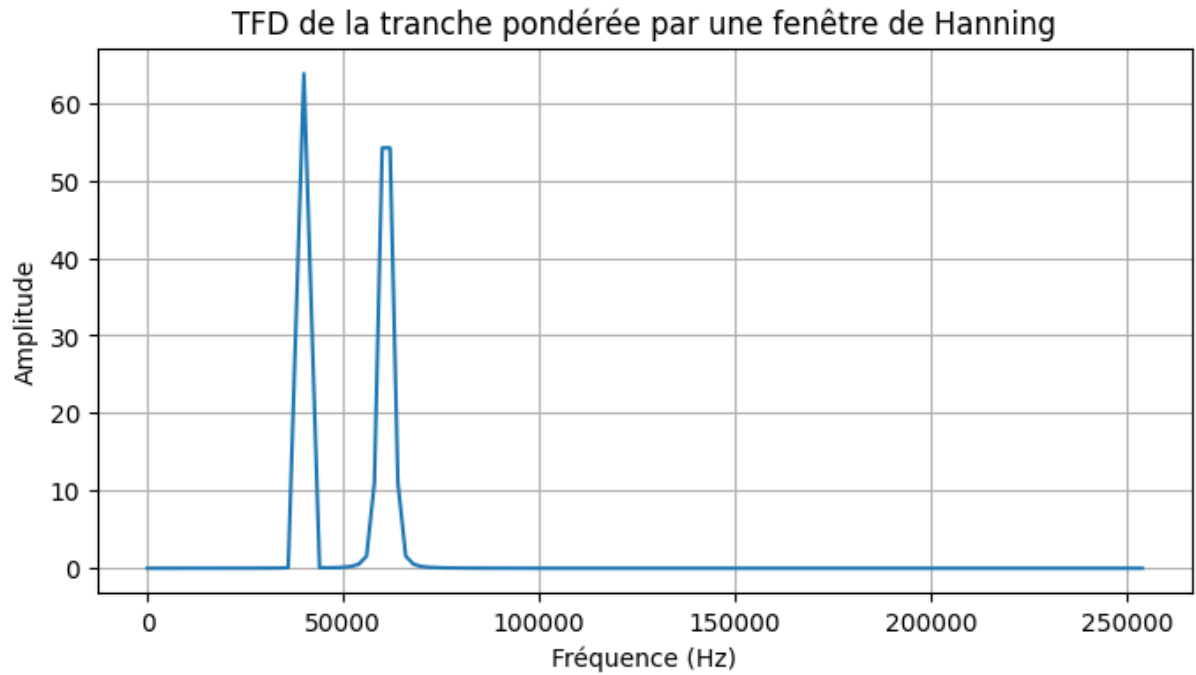


Figure 21: TFD avec fenêtre de Hanning

3.4.2 Effets de quelques fenêtres

Dans les trois cas, les deux fréquences dominantes du signal (~ 995 Hz et ~ 1200 Hz) sont bien détectées, mais on observe des différences marquées d'amplitude liées à la variation entre A_0 et A_1 . On note également une légère variation dans la forme du pic autour de 995 Hz selon le type de fenêtre utilisée.

L'application d'une fenêtre de Hanning permet de réduire efficacement les lobes secondaires par rapport à une fenêtre rectangulaire, tout en maintenant une bonne résolution en fréquence, ce qui facilite la distinction entre les deux composantes spectrales. La fenêtre de Hamming atténue encore davantage les lobes secondaires, mais cela se fait au prix d'un élargissement du lobe principal, ce qui peut compliquer la séparation des deux fréquences si elles sont rapprochées. De son côté, la fenêtre de Blackman offre la meilleure atténuation des lobes secondaires, mais sa résolution fréquentielle est la plus faible.

En somme, le choix de la fenêtre constitue un compromis entre la capacité à séparer précisément les fréquences (résolution fréquentielle) et la réduction des fuites spectrales (lobes secondaires). Ce compromis est particulièrement crucial ici pour réussir à mettre en évidence la faible composante à 1200 Hz face à une composante dominante à 995 Hz.

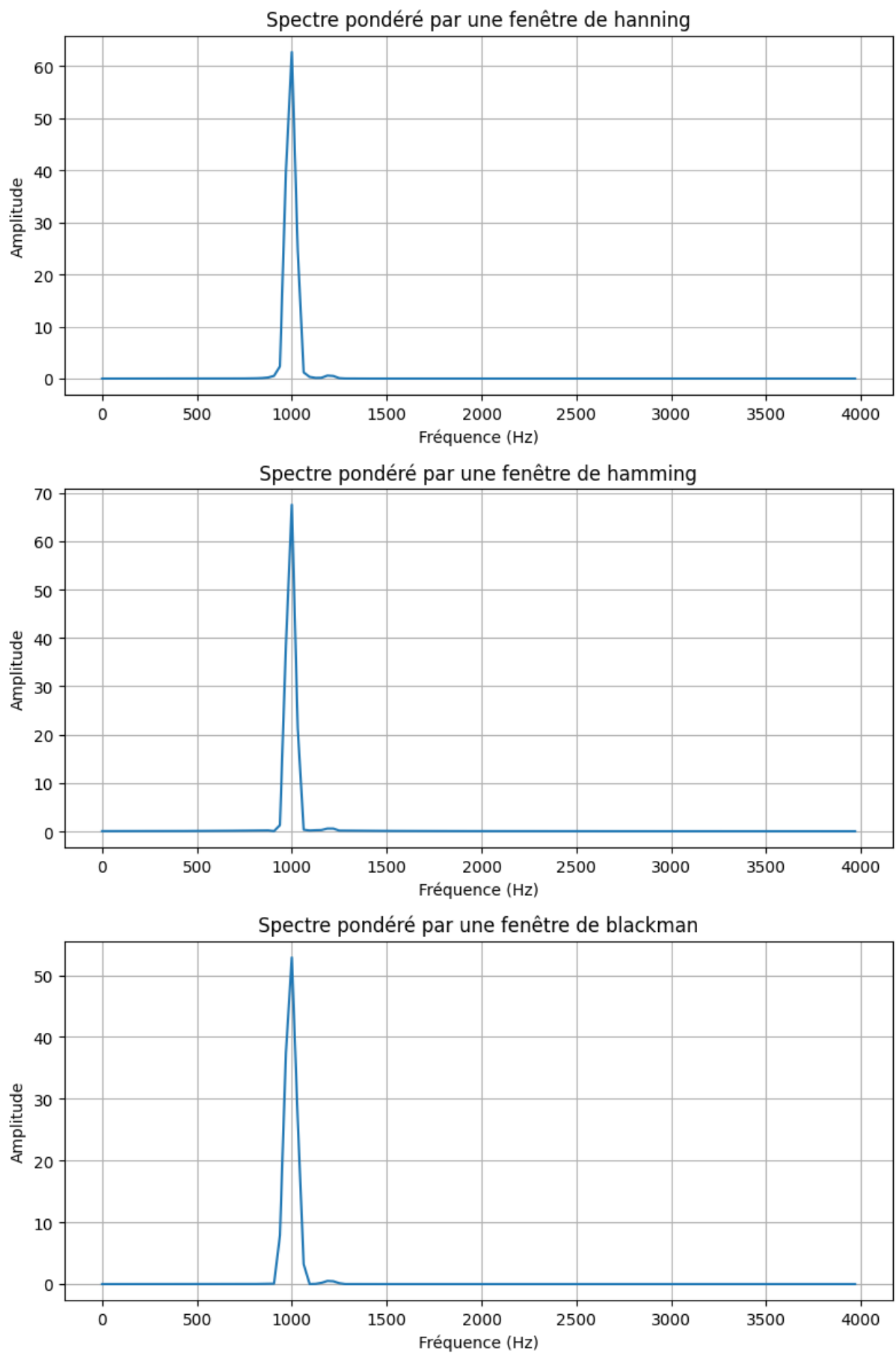


Figure 22: TFD avec différentes fenêtres

Dans ce cas, on analyse un signal de courte durée, composé de 256 échantillons, contenant deux fréquences

proches : $f_0 = 995$ Hz et $f_1 = 1200$ Hz. La résolution fréquentielle théorique, sans zero-padding, est donnée par :

$$\Delta f = \frac{f_e}{N} = \frac{8000}{256} = 31,25 \text{ Hz}$$

Cela signifie que les fréquences ne peuvent être détectées qu'à des positions multiples de 31,25 Hz (par exemple : 968,75 Hz, 1000 Hz, 1031,25 Hz, etc.), ce qui est insuffisant pour distinguer précisément les composantes situées à 995 Hz et 1200 Hz.

L'ajout de *zero-padding*, par exemple en prolongeant le signal à 1024 points, n'apporte aucune information supplémentaire sur le contenu fréquentiel réel du signal, mais permet de raffiner la grille fréquentielle. La nouvelle résolution de l'axe fréquentiel devient :

$$\Delta f_{\text{interp}} = \frac{f_e}{1024} = \frac{8000}{1024} = 7,8125 \text{ Hz}$$

Cette interpolation plus fine permet d'observer les pics spectraux de manière plus précise (autour de 995 Hz et 1200 Hz), même si la capacité à les distinguer — c'est-à-dire la résolution effective — reste inchangée. En résumé, le zero-padding améliore la lecture du spectre (interpolation), mais n'améliore pas la séparation entre fréquences proches (résolution).

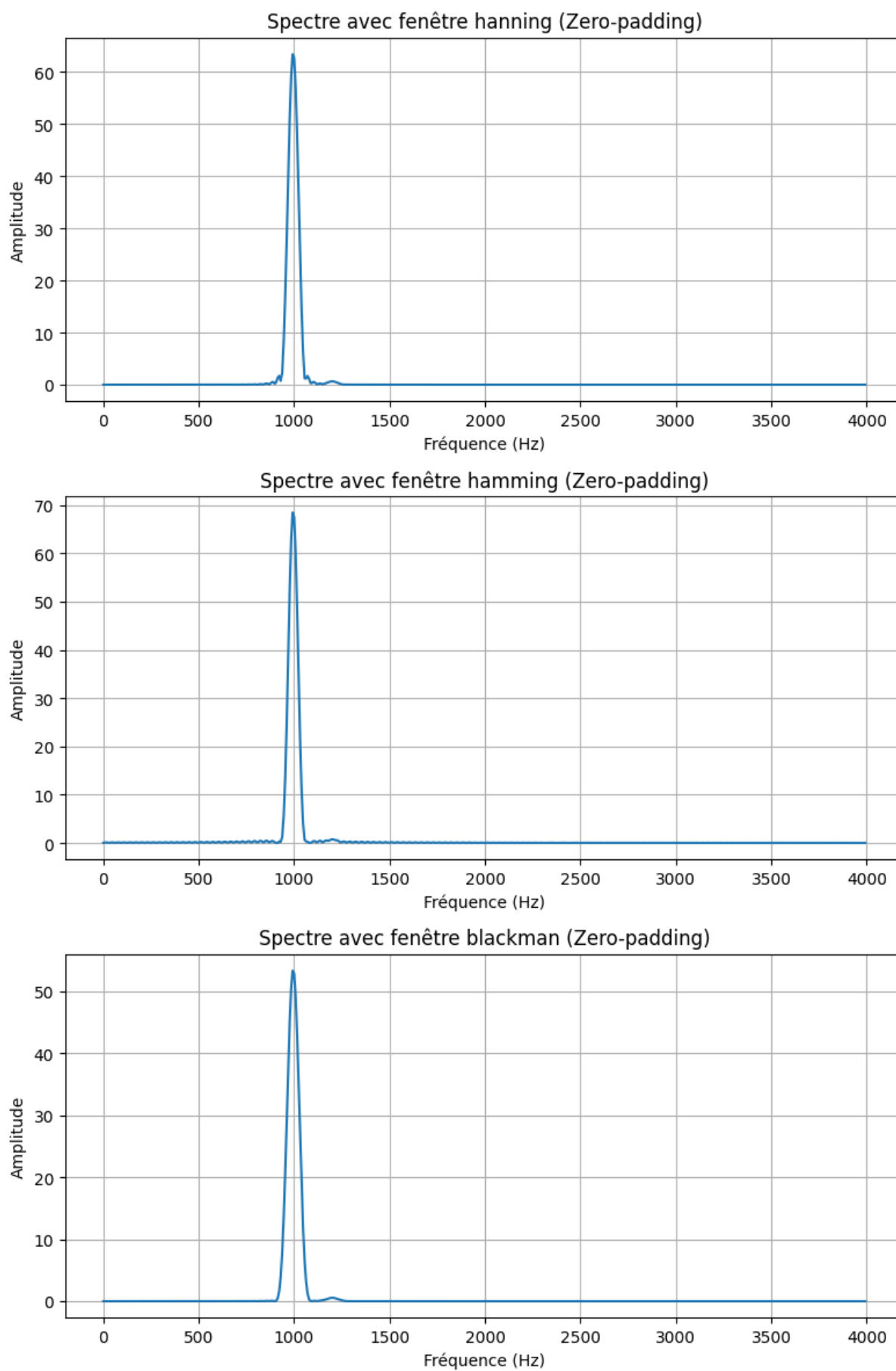


Figure 23: TFD avec différentes fenêtres + zero-padding

4 Partie 4: Application : détection de pitch

4.1 Code python

On a fait le code suivant :

```
import scipy.io.wavfile as wav

def is_voiced(trame, seuil_energie):

    energie = np.sum(trame ** 2) / len(trame)
    return energie > seuil_energie

def autocorrelation_pitch(trame, fs, fmin, fmax):

    # Soustraire la moyenne pour centrer
    trame = trame - np.mean(trame)
    corr = np.correlate(trame, trame, mode='full')
    corr = corr[len(corr)//2:] # ne garder que la partie positive
    # Determiner les bornes de lag correspondant a fmin et fmax
    lag_min = int(fs / fmax)
    lag_max = int(fs / fmin)
    if lag_max >= len(corr):
        lag_max = len(corr) - 1
    # Chercher le pic dans la plage de lag
    segment = corr[lag_min:lag_max+1]
    if len(segment) == 0:
        return None
    pic_indice = np.argmax(segment) + lag_min
    if corr[pic_indice] <= 0:
        return None
    # Convertir le lag en frequence
    freq = fs / pic_indice
    return freq

def freq_en_midi(freq):
    #Convertit une frequence en numero de note MIDI.
    if freq is None or freq <= 0:
        return None
    return int(np.round(69 + 12 * np.log2(freq / 440.0)))

def detect_pitch_sequence(
    wav_filename,
    duree_trame=0.030,
    pitch_range=(80.0, 400.0),
    min_note_duree=0.1
):
    """
    Fonction principale de detection de pitch et de notes.

    Parametres :
    - wav_filename : chemin vers le fichier .wav
    - duree_trame : duree (en secondes) de chaque trame
    - pitch_range : (fmin, fmax) domaine de recherche du pitch en Hz
    - min_note_duree : duree minimale d'une note en secondes

    Retourne :
    - matrice numpy de shape (N, 3) avec colonnes [temps, frequence, note_MIDI]
    """
    # Lecture du signal
    fs, signal_data = wav.read(wav_filename)

    # Si le signal est stereo, le convertir en mono
    if signal_data.ndim == 2:
        signal_data = signal_data.mean(axis=1)
```

```

# Normaliser le signal entre -1 et 1 si necessaire
if signal_data.dtype != np.float32 and signal_data.dtype != np.float64:
    max_val = np.iinfo(signal_data.dtype).max
    signal_data = signal_data.astype(np.float32) / max_val

# Parametres de tramage
trame_len = int(duree_trame * fs)
hop_len = trame_len # trames sans recouvrement
num_trames = int(np.floor(len(signal_data) / hop_len))

# Calcul du seuil d'energie (par exemple, 10% de l'energie max d'une trame)
energies = []
for i in range(num_trames):
    start = i * hop_len
    trame = signal_data[start : start + trame_len]
    energies.append(np.sum(trame ** 2) / len(trame))
seuil_energie = 0.05 * np.max(energies)

# Allocation des resultats
temps = np.zeros(num_trames)
freqs = np.zeros(num_trames)
midis = np.zeros(num_trames, dtype=int)

fmin, fmax = pitch_range

# Parcours des trames
for i in range(num_trames):
    start = i * hop_len
    trame = signal_data[start : start + trame_len]
    temps[i] = (start + trame_len / 2) / fs # temps au centre de la trame

    if is_voiced(trame, seuil_energie):
        freq = autocorrelation_pitch(trame, fs, fmin, fmax)
        freqs[i] = freq if freq is not None else 0.0
        midi = freq_en_midi(freq) if freq is not None else 0
        midis[i] = midi if midi is not None else 0
    else:
        freqs[i] = 0.0
        midis[i] = 0

# Detection des notes : regrouper les trames de meme note MIDI consecutives
min_trames_par_note = int(np.ceil(min_note_duree / duree_trame))
i = 0
while i < num_trames:
    if midis[i] > 0:
        j = i + 1
        while j < num_trames and midis[j] == midis[i]:
            j += 1
        longueur = j - i
        if longueur < min_trames_par_note:
            # Pas assez long : considerer comme bruit -> zero
            freqs[i:j] = 0.0
            midis[i:j] = 0
        i = j
    else:
        i += 1

# Construire la matrice resultat
result = np.column_stack((temps, freqs, midis))
return result

# On applique le programme sur tous les signaux audio qui sont pr sent sur le
fichier du projet
for file in os.listdir("./"):
    if file.endswith(".wav") or file.endswith(".WAV"):

```

```

print(f"Traitement du fichier : {file}")

result = detect_pitch_sequence(
    f"./{file}",
    frame_duration=0.025,
    pitch_range=(80.0, 300.0),
    min_note_duration=0.1
)
np.savetxt(f"./partie4_output/pitch_detection_{file[:-4]}.csv", result,
    delimiter=",",
    header="time,frequency,midi", comments='')

```

Chaque étape du code est commentée mais j'ajoute certains commentaires ici: α (energy_factor) : 0,005–0,05 : à adapter en fonction de l'amplitude relative du signal et du bruit de fond.

4.1.0.1 Conclusion Cf le dossier partie4output Le programme réalisé permet de détecter automatiquement les notes présentes dans un fichier audio au format **.wav**. Il s'appuie sur une segmentation du signal en trames, une détection voisée basée sur l'énergie, puis une estimation de la fréquence fondamentale par autocorrélation. Ces fréquences sont ensuite converties en notes MIDI, et les segments trop courts sont filtrés pour éliminer les détections non pertinentes.

La fonction principale respecte les contraintes demandées (paramètres d'entrée et sortie sous forme de matrice [temps, fréquence, note MIDI]). Le traitement est robuste sur les fichiers contenant une voix ou une mélodie claire, mais peut être sensible au bruit ou aux silences prolongés.

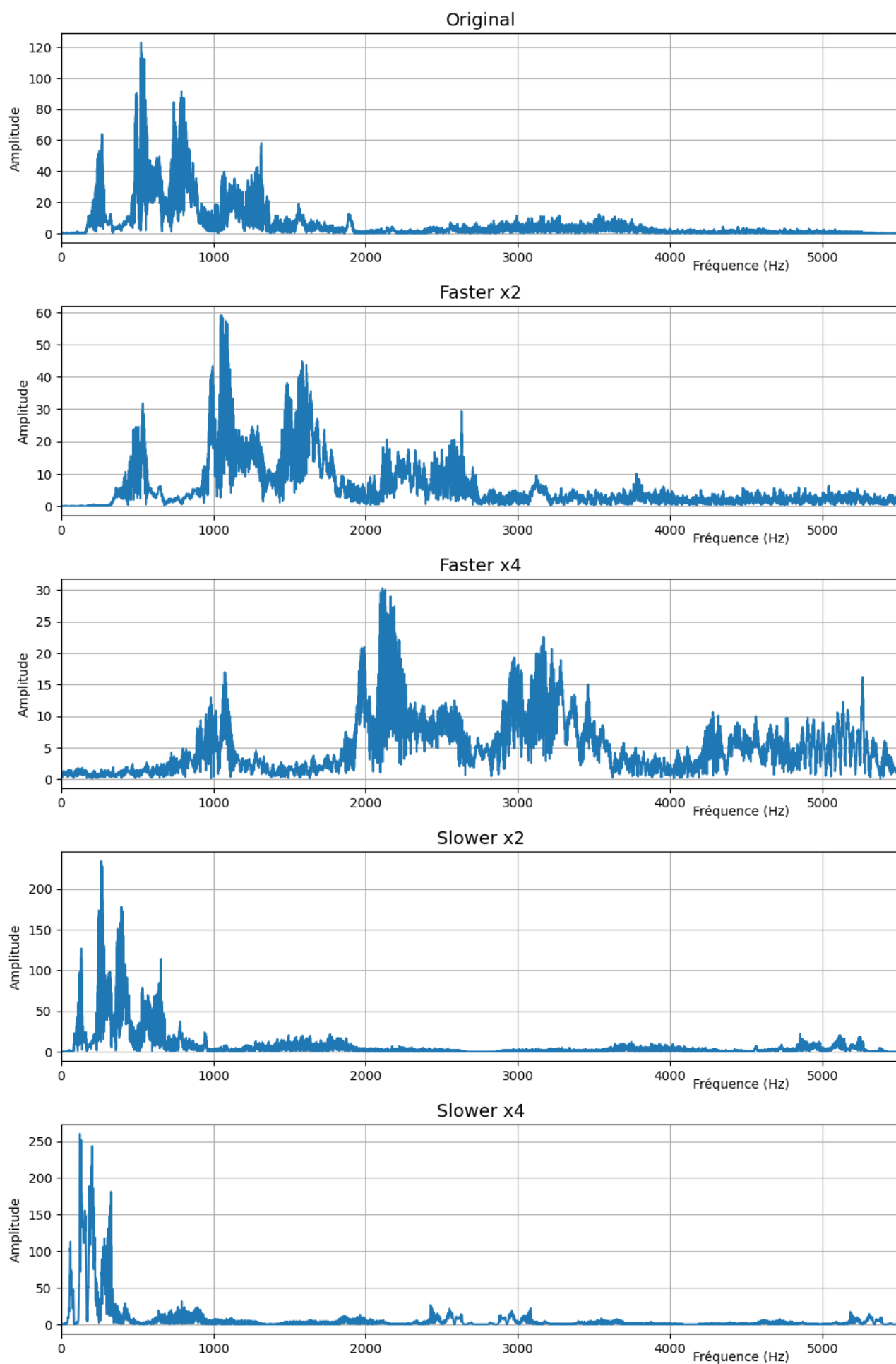


Figure 17: FFT cadence modifiée